

Module 2 : Syntax Analysis - Morphological Analysis

Dr. Priya R L

Faculty Incharge for NCMPE64 (Natural Language Processing & Generative AI)

Department of Computer Engineering

VES Institute of Technology, Mumbai

Agenda

- Syntax Analysis - Introduction
- POS tagging
- Tagset for English
- Generative Models
- HMM
- CRF
- Parsers

Stochastic Tagging

- The use of probabilities in tags is quite old. Stochastic taggers use **probabilistic and statistical information** to assign tags to words.
- The model that includes **frequency or probability (statistics) can be called stochastic.**
- These taggers might use **'tag sequence probabilities', 'word frequency measurements'** or a combination of both.
- The tag encountered most frequently in the training set is the one assigned to an ambiguous instance of that word (**word frequency measurements**).

Stochastic Tagging

- The best tag for a given word is determined by the probability that it occurs with the n previous tags (**tag sequence probabilities**)
- **Resolves the ambiguity** by computing the probability of a given word (or the tag).
- The problem with this approach is that it can come up with **sequences of tags for sentences that are not acceptable according to the grammar rules of a language.**

Stochastic POS Tagger Approaches

Word Frequency Approach

- In this approach, the stochastic taggers disambiguate the words based on the probability that a word occurs with a particular tag.
- We can also say that the **tag encountered most frequently with the word in the training set is the one assigned to an ambiguous instance of that word.**
- The main issue with this approach is that it may yield **inadmissible sequence** of tags.

Tag Sequence Probabilities

- Here the tagger calculates the **probability of a given sequence of tags occurring.**
- It is also called **n-gram approach**. It is called so because the best tag for a given word is determined by the probability at which it occurs with the n previous tags.

Stochastic POS Tagger Approaches

- **Unigram** approach assigns each word to its most common tag and consider one word at a time. $P(t_i/w_i) = \text{freq}(w_i/t_i) / \text{freq}(w_i)$
- Here Probability of tag given word is computed by frequency count of word in given tag divided by frequency count of that particular word
- **Bigram** approach is based on preceding tag i.e. it take two tags: the preceding tag and current tag into account. $P(t_i/w_i) = P(w_i/t_i) \cdot P(t_i/t_{i-1})$
- Here $P(w_i/t_i)$ is the probability of current word in given current tag and $P(t_i/t_{i-1})$ is the probability of a current tag in given the previous tag

Stochastic POS Tagger Approaches

- **Trigram** is based on previous two tags. $P(ti/wi) = P(wi/ti) \cdot P(ti/ti-2, ti-1)$, Where ti denotes tag sequence and wi denote word sequence.
- $P(wi/ti)$ is the probability of current word given current tag.
- Here, $P(ti/ti-2, ti-1)$ is the probability of a current tag given the previous two tags.

Properties of Stochastic POS Tagging

- This POS tagging is based on the **probability of tag occurring**.
- It requires **training corpus**
- There would be no probability for the words that **do not exist in the corpus**.
- It **uses different testing corpus** (other than training corpus).
- It is the simplest POS tagging because it chooses **most frequent tags** associated with a word in training corpus.

What are Generative Models?

- A generative model learns the joint probability of data and labels: $P(X, Y)$
- Where, X : observed data (Words), Y : Hidden Structure (tags, Parse trees)
- **Key Idea:** Generate observations from hidden states and model how language is produced.

Examples

- HMM (classic generative sequence model)
- Probabilistic Context-Free Grammar (PCFG)

Stochastic POS Tagging : HMM Tagger

- **Intuition:** Pick the most likely tag based on context
- **Maximize** the formula using a HMM : $P(\text{word}|\text{tag}) \times P(\text{tag}|\text{previous } n \text{ tags})$
- **Observe:** $W = w_1, w_2, \dots, w_n$
- **Hidden:** $T = t_1, t_2, \dots, t_n$
- **Goal:** Find POS tags that generate a sequence of words, i.e., look for most probable sequence of tags T underlying the observed words W
- We cannot determine the exact sequence of tags that generated and calculate using $t = \text{argmax } P(w, t)$ and it is **based on the Markovian assumption that the current tag depends only on the previous n tags.**
- **Use transition probability** (i.e. forward tag and backward tags).
- $P(t_i/w_i) = P(t_i/t_{i-1}) \cdot P(t_{i+1}/t_i) \cdot P(w_i/t_i)$
- $P(t_i/t_{i-1})$ is the probability of current tag given previous tag

Hidden Markov Model (HMM) Tagger

- Finding the best sequence of tags $(t_1, t_2, \dots, t_n)$ that corresponds to the sequence of observations $(w_1, w_2, \dots, w_n)$
- Probabilistic View:
 - Considering all possible sequences of tags
 - Choosing the tag sequence from this universe of sequences, which is most probable given the observation sequence.

$$\hat{t}_1^n = \operatorname{argmax}_{t_1^n} P(t_1^n | w_1^n)$$

HMM Tagger: Using Bayes Rule

$$\hat{t}_1^n = \underset{t_1^n}{\operatorname{argmax}} P(t_1^n | w_1^n)$$

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

$$P(t_1^n | w_1^n) = \frac{P(w_1^n | t_1^n) \cdot P(t_1^n)}{P(w_1^n)}$$

$$\hat{t}_1^n = \underset{t_1^n}{\operatorname{argmax}} \underbrace{P(w_1^n | t_1^n)}_{\text{likelihood}} \cdot \underbrace{P(t_1^n)}_{\text{prior probability}}$$

HMM Tagger: Using Markov Assumption

$$\hat{t}_1^n = \operatorname{argmax}_{t_1^n} P(w_1^n | t_1^n) \cdot P(t_1^n)$$

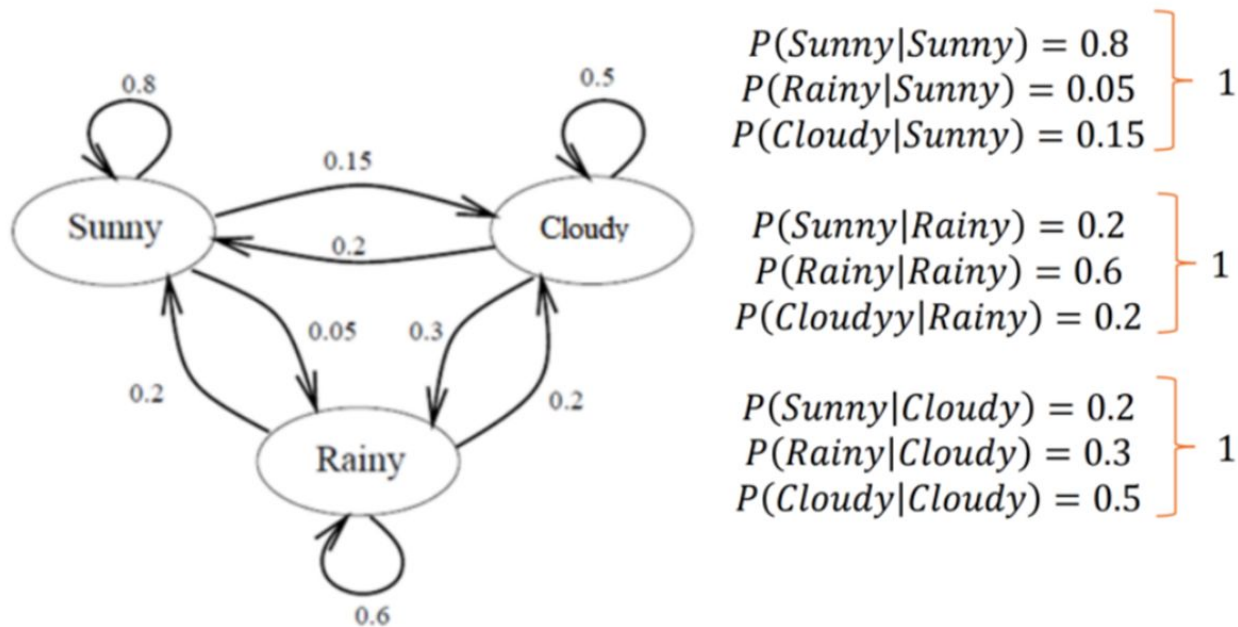
$$P(w_1^n | t_1^n) \simeq_{i=1}^n \prod P(w_i | t_i) \quad (\text{it depends only on its POS tag and independent of other words})$$

$$P(t_1^n) \simeq_{i=1}^n \prod P(t_i | t_{i-1}) \quad (\text{it depends only on the previous POS tag, thus, bigram})$$

$$\hat{t}_1^n = \operatorname{argmax}_{t_1^n} \prod_{i=1}^n P(w_i | t_i) \cdot P(t_i | t_{i-1})$$

HMM Tagger: Exercise 1

Given that today is Sunny, What's the probability that tomorrow is Sunny and next day is Rainy? Solve the problem using Markov Model.



HMM Tagger: Solution 1

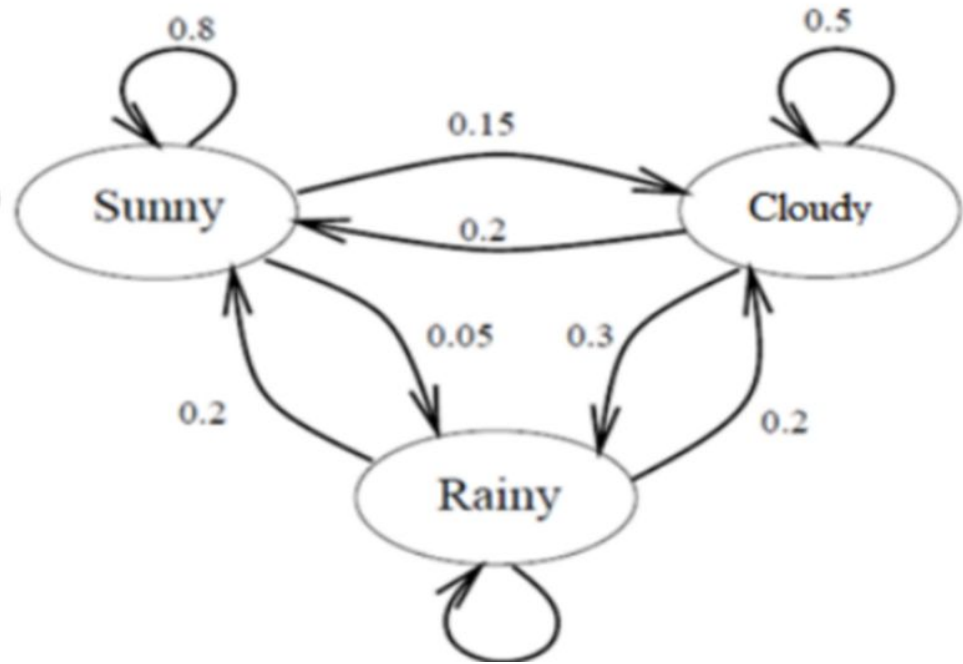
$$P(q_2, q_3 | q_1) = P(q_2 | q_1) P(q_3 | q_1, q_2)$$

$$= P(q_2 | q_1) P(q_3 | q_2)$$

$$= P(\text{Sunny} | \text{Sunny}) P(\text{Rainy} | \text{Sunny})$$

$$= (0.8)(0.05)$$

$$= 0.04$$



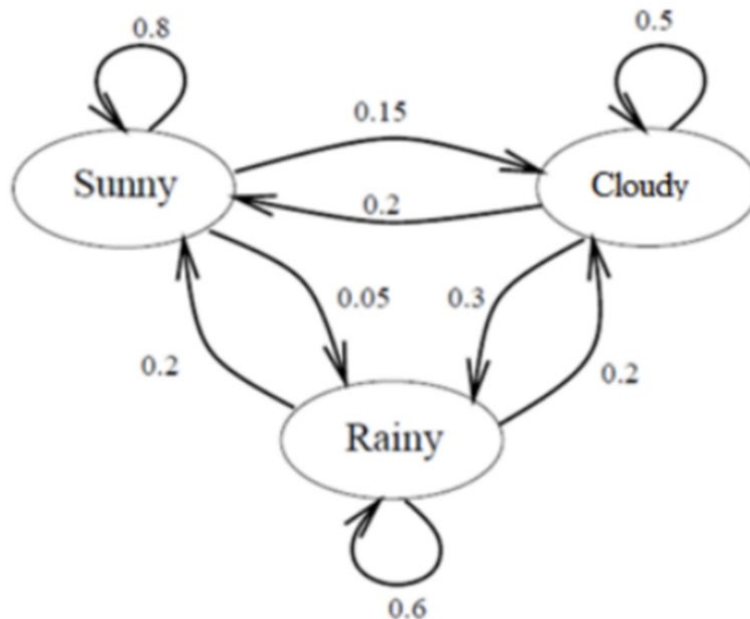
HMM Tagger: Exercise 2

- Exercise 2: Assume that yesterday's weather was Rainy, and today is Cloudy, what is the probability that tomorrow will be Sunny?

$$P(q_3|q_1, q_2) = P(q_3|q_2)$$

$$= P(\text{Sunny}|\text{Cloudy})$$

$$= 0.2$$



Two Probabilities

- The tag transition probabilities: $P(t_i|t_{i-1})$
 - Finding the likelihood of a tag to proceed by another tag
 - Similar to the normal bigram model

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

Two Probabilities

- The word likelihood probabilities: $P(w_i|t_i)$
 - Finding the likelihood of a word to appear given a tag

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$



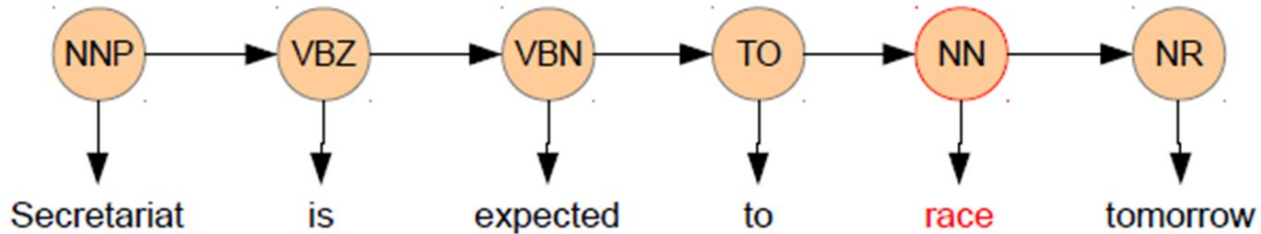
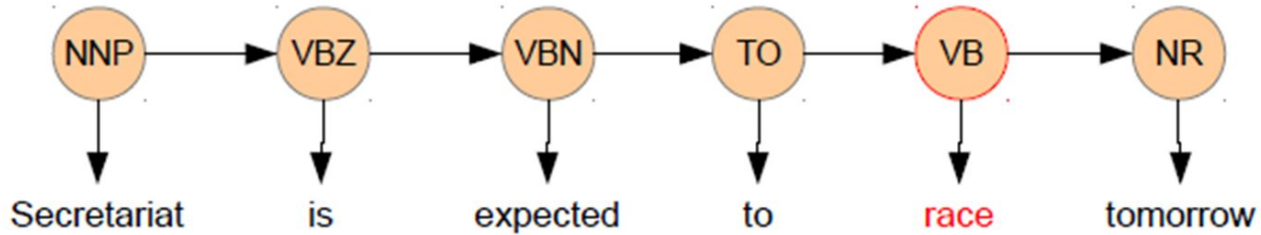
Ambiguity

Secretariat_[NNP] is_[VBZ] expected_[VBN] to_[TO] race_[VB] tomorrow_[NR] .

People_[NNS] inquire_[VB] the_[DT] reason_[NN] for_[IN] the_[DT] race_[NN] .

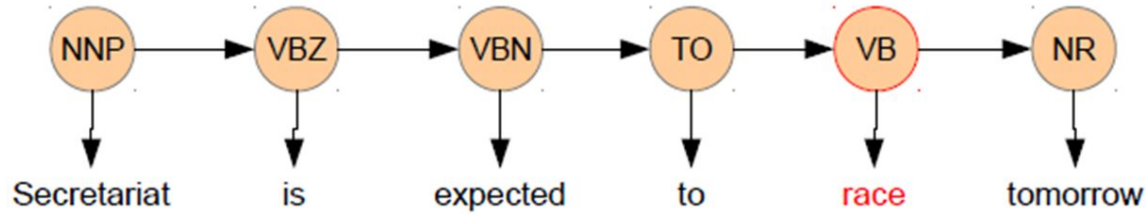
Ambiguity

Secretariat_[NNP] is_[VBZ] expected_[VBN] to_[TO] race_[VB] tomorrow_[NR] .



Ambiguity

Secretariat_[NNP] is_[VBZ] expected_[VBN] to_[TO] race_[VB] tomorrow_[NR] .



$$P(\text{VB}|\text{TO}) = 0.83$$

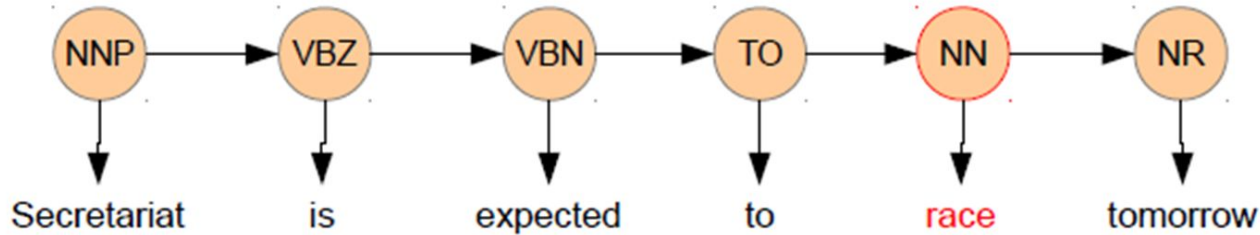
$$P(\text{race}|\text{VB}) = 0.00012$$

$$P(\text{NR}|\text{VB}) = 0.0027$$

$$P(\text{VB}|\text{TO}).P(\text{NR}|\text{VB}).P(\text{race}|\text{VB}) = 0.00000027$$

Ambiguity

Secretariat_[NNP] is_[VBZ] expected_[VBN] to_[TO] race_[VB] tomorrow_[NR] .



$$P(\text{NN}|\text{TO}) = 0.00047$$

$$P(\text{race}|\text{NN}) = 0.00057$$

$$P(\text{NR}|\text{NN}) = 0.0012$$

$$P(\text{NN}|\text{TO}).P(\text{NR}|\text{NN}).P(\text{race}|\text{NN}) = 0.00000000032$$

Ambiguity

- $P(\text{VB}|\text{TO})P(\text{NR}|\text{VB})P(\text{race}|\text{VB}) = .00000027$
- $P(\text{NN}|\text{TO})P(\text{NR}|\text{NN})P(\text{race}|\text{NN}) = .00000000032$
- So we (correctly) choose the verb reading.

Ambiguity

- $P(\text{VB}|\text{TO})P(\text{NR}|\text{VB})P(\text{race}|\text{VB}) = .00000027$
- $P(\text{NN}|\text{TO})P(\text{NR}|\text{NN})P(\text{race}|\text{NN}) = .00000000032$
- So we (correctly) choose the verb reading.

HMM : Pros & Cons...

Strengths

- Simple and efficient
- Works well with limited data
- Strong theoretical foundation

Weaknesses

- Strong **independence assumptions** - Assumes features don't influence each other & misses real relationships
- **Poor at using rich features** - Can't use complex contextual info
- Struggles with **long-range dependencies** - Poor understanding of long text

Why Conditional Random Field (CRF)?

- HMMs model $P(X, Y)$: Models that are globally conditioned on the observation sequence - (Lafferty et al 2001)
- We want to model $P(Y | X)$ directly : define distribution $P(Y|X)$ of tag sequence given word sequence
- Avoid unrealistic independence assumptions
- The labels can depend on past and future observations

CRFs are discriminative sequence models

- CRFs is a popular probabilistic method for structured prediction
- Common formula behind many applications is the ability to predict multiple variables that depend on each other.

Why do we need CRFs?

- Consider an application, where we want the ability to predict multiple variables that depend on each other. For Ex.: the performance of a sports team depends on the health of each member of the team. The health of each member might get affected by the travel schedule of the team. The results of the game might affect the morale of the team. The morale, in turn, might affect the health. Here, there are multiple variables that depend on each other intricately. CRFs are very useful in modeling these problems.

CRF Probability Definition

- For input sequence X and label sequence Y :
 - $P(Y | X) = (1/Z(X)) \exp(\sum \lambda_i f_i(Y, X))$
- Where:
- f_i = feature functions
- λ_i = learned weights
- $Z(X)$ = normalization factor

CRF Feature Functions

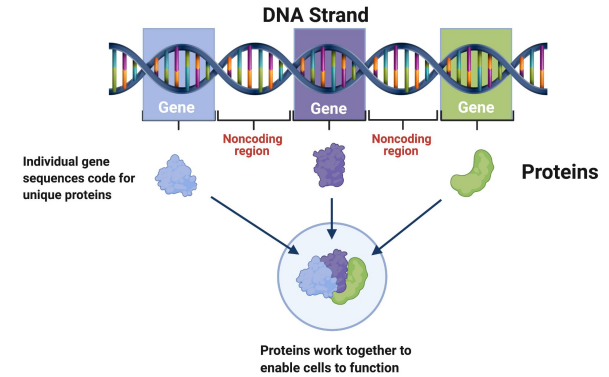
Features can use:

- Current word
- Surrounding words
- Capitalization
- Prefixes / suffixes
- POS tags

Example feature: $f(y_i, x_i) = 1$ if x_i is capitalized and $y_i = \text{NNP}$

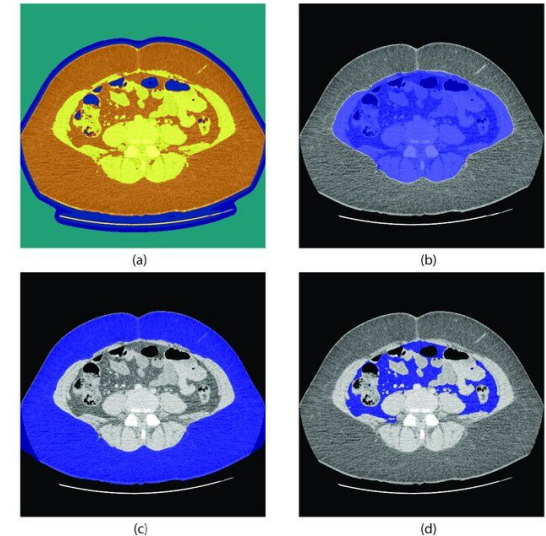
Applications..

- Segmenting genes in a strand of DNA
- Extracting Syntax from natural language text
- Classifying regions of an image
- Estimating the score in a strategic game.



	Player 2 chooses Up	Player 2 chooses Down
Player 1 chooses Top	0,0	2,1
Player 1 chooses Bottom	1,2	0,0

Payoff Matrix



Importance of Context

Problem Statement:

Given a sentence, we have to label the words as Nouns / Verbs / Others.

For example:

X = I play Cricket

Y = [Noun, Verb, Noun]

X = Fish don't sleep

Y = [Noun, Other, Verb]

X = I like to fish

Y = [Noun, Other, Other, Verb]

Notice the labelling of the word "Fish" here.

It can be both Noun or Verb depending upon the context.

Hence, relative ordering of tokens in sequence is very important for labeling.

Differences bet Generative & Discriminative Models

Generative Models

Joint distribution: $P(x,y)$

Use Baye's Theorem to predict $P(y|x)$

Examples

Naive Multiclass
Hidden Markov Models

Linear functions

Easy to train but less accurate

Discriminative Models

Conditional distribution: $P(y|x)$

Can directly predict $P(y|x)$

Examples

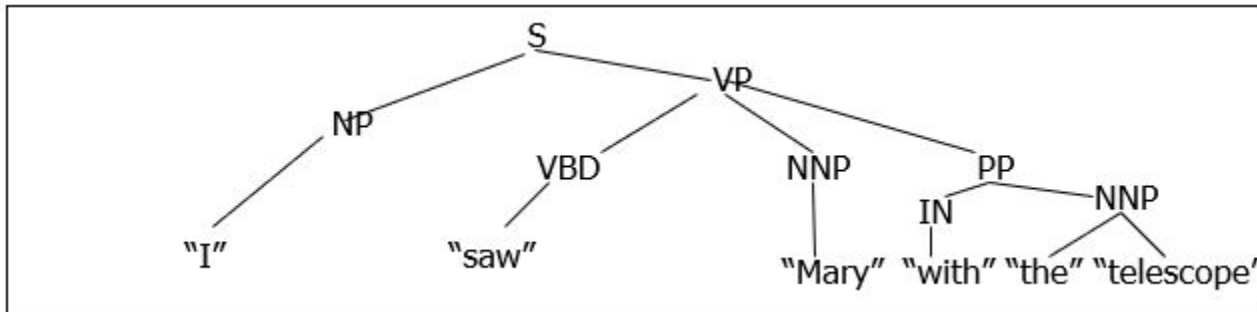
Logistic Regression
Conditional Random Fields

Exponential functions

Hard to train but more accurate

Parsers

- **Syntactic parsing** is the task of **assigning a parse tree to a sentence** corresponding to its most likely interpretation.
- **Existing approaches:**
 - Hand-crafted rule-based heuristic methods
 - Probabilistic generative models
 - Conditional probabilistic discriminative models
 - Discriminative ranking models



Basic Parsing Techniques

- **Parsing** is the process of analyzing a sentence by taking apart word-by-word and determining its structure from its constituent parts and subparts.
- The **structure of the sentence can be represented with a syntactic tree** known as **parse tree**.
- **Two techniques of Parsing:**
 - Top-down parsing
 - Bottom-Up Parsing

Why should we care for Syntax?

- Grammar checkers, Question answering
- Information extraction, Machine translation

Key ideas of Syntax

- **Constituency** – group of words behave as a single unit or phrase. Eg. :
Noun Phrase
- **Sub-categorization and dependencies** – refer to certain kinds of relations between words and phrases. Ex.: You said [Delhi has a flight]
- **Grammatical relations** – formalization of ideas from traditional grammar about Subjects and Objects in the sentence. E.g., : She ate a VadaPav
(Here, She is a subject & a VadaPav is a Object)



Basic Parsing Techniques

- **Top-down Parsing :** begins with the start symbol 'S' and derives the tree using the production rules of the given sentence.
- **Bottom-Up Parsing:** Begins with the actual words appearing in the sentence and moves up to derive the starting symbol, 'S'.

Example : CFG

$R = \{$

$S \rightarrow NP VP$

$S \rightarrow Aux NP VP$

$S \rightarrow VP$

$NP \rightarrow Det NOM$

$NOM \rightarrow Noun$

$NOM \rightarrow Noun NOM$

$VP \rightarrow Verb$

$VP \rightarrow Verb NP$

$Det \rightarrow that \mid this \mid a \mid the$

$Noun \rightarrow book \mid flight \mid meal \mid man$

$Verb \rightarrow book \mid include \mid read$

$Aux \rightarrow does$

$G = \langle T, N, S, R \rangle$

$T = \{that, this, a, the, man, book, flight, meal, include, read, does\}$

$N = \{S, NP, NOM, VP, Det, Noun, Verb, Aux\}$

$S = S$

Top-Down Parsing Techniques

Let us generate the first sentence

$S \rightarrow NP VP$

$\rightarrow ART N VP$

$\rightarrow the N VP$

$\rightarrow the boy VP$

$\rightarrow the boy V NP$

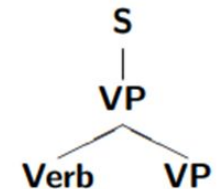
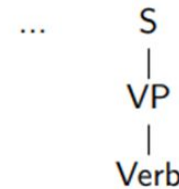
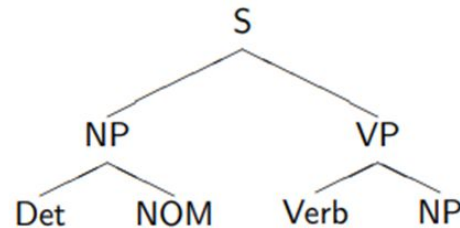
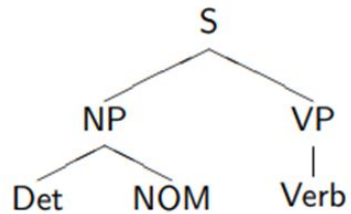
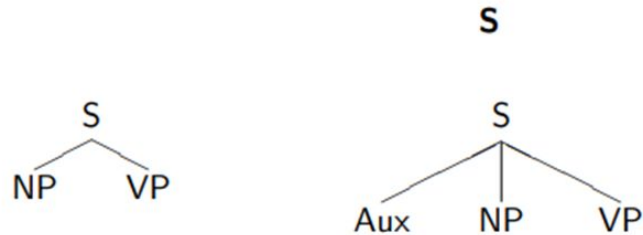
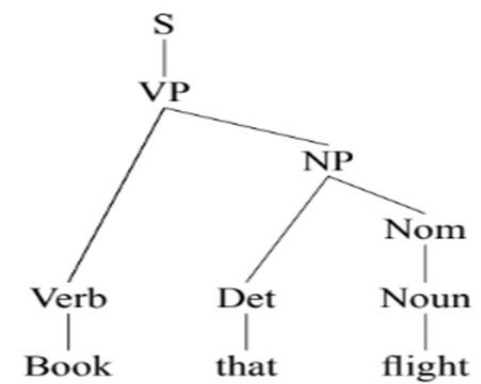
$\rightarrow the boy ate NP$

$\rightarrow the boy ate ART N$

$\rightarrow the boy ate a N$

$\rightarrow the boy ate a cake$

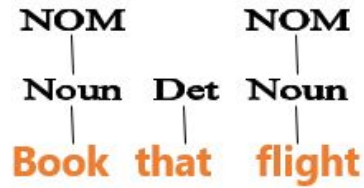
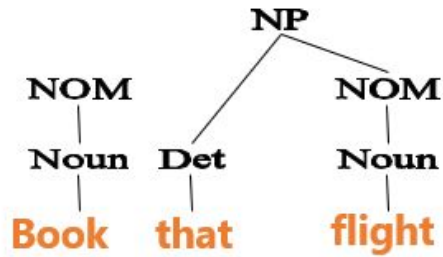
Top-Down Parsing



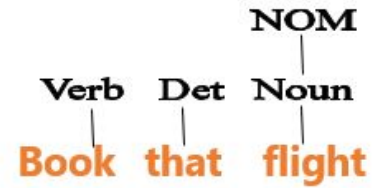
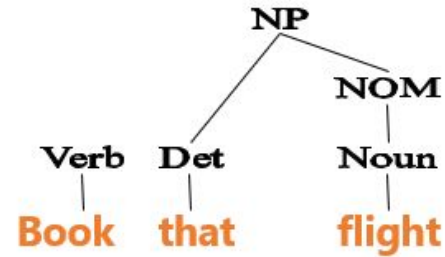
Bottom-Up Parsing Techniques

- $\rightarrow$ the boy ate a cake
- $\rightarrow$ the boy ate a N
- $\rightarrow$ the boy ate ART N
- $\rightarrow$ the boy ate NP
- $\rightarrow$ the boy V NP
- $\rightarrow$ the boy VP
- $\rightarrow$ the N VP
- $\rightarrow$ ART N VP
- $\rightarrow$ NP VP
- $\rightarrow$ S

Bottom-Up Parsing Techniques



Book that flight



Top-Down Vs Bottom-Up

Top-down

- Only searches for trees that can be answers
- But suggests trees that are not consistent with the words
- Guarantees that tree starts with S as root
- Does not guarantee that tree will match input words

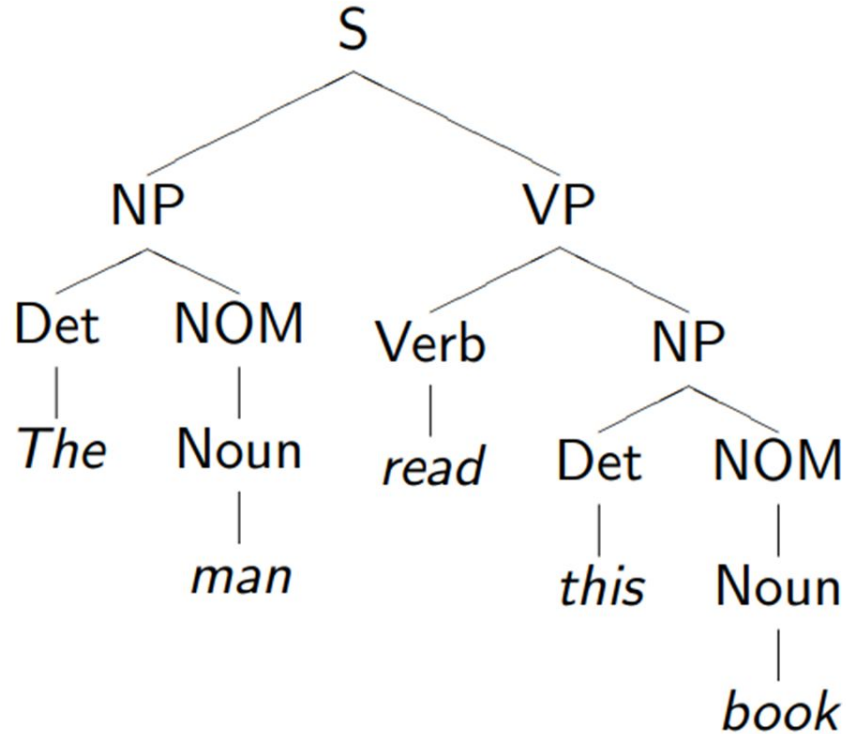
Bottom-up

- Only forms trees consistent with the words
- Suggest trees that make no sense globally
- Guarantees that tree matches input words
- Does not guarantee that parse tree will lead to S as a root

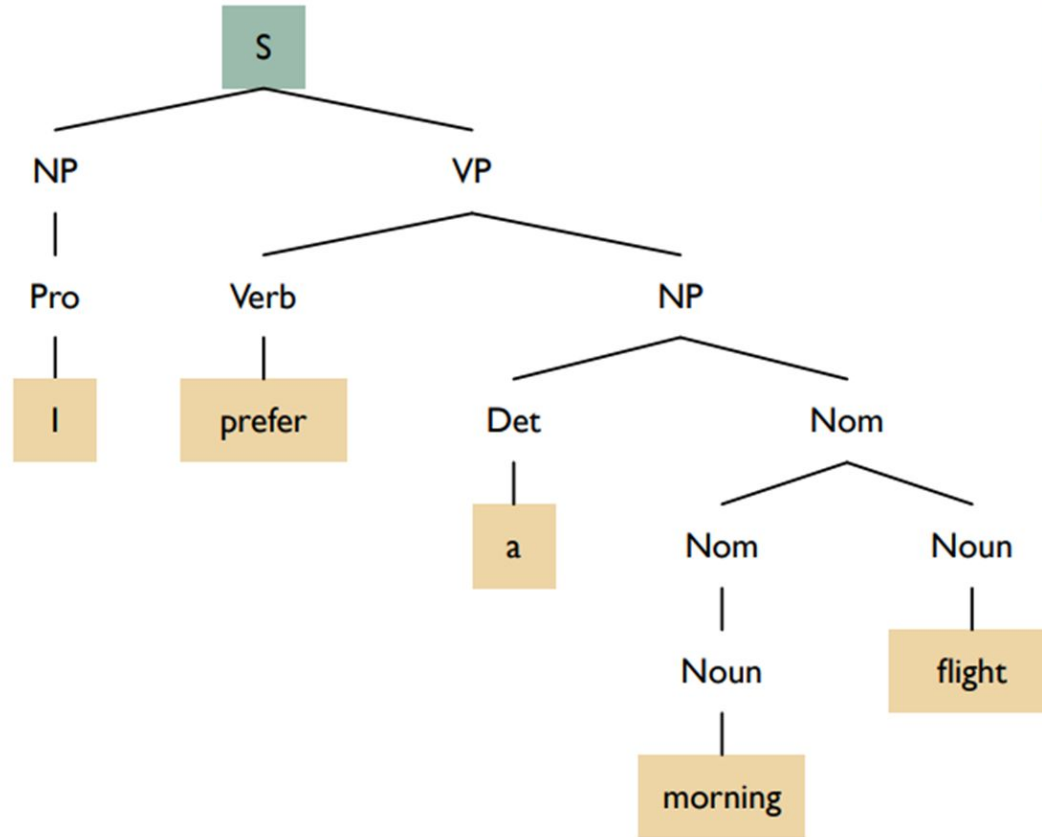
Example : CFG

$S \rightarrow NP VP$
 $\rightarrow Det NOM VP$
 $\rightarrow The NOM VP$
 $\rightarrow The Noun VP$
 $\rightarrow The man VP$
 $\rightarrow The man Verb NP$
 $\rightarrow The man read NP$
 $\rightarrow The man read Det NOM$
 $\rightarrow The man read this NOM$
 $\rightarrow The man read this Noun$
 $\rightarrow The man read this book$

Parse Tree



Parse Tree: Example



root (top)

leaves (bottom)

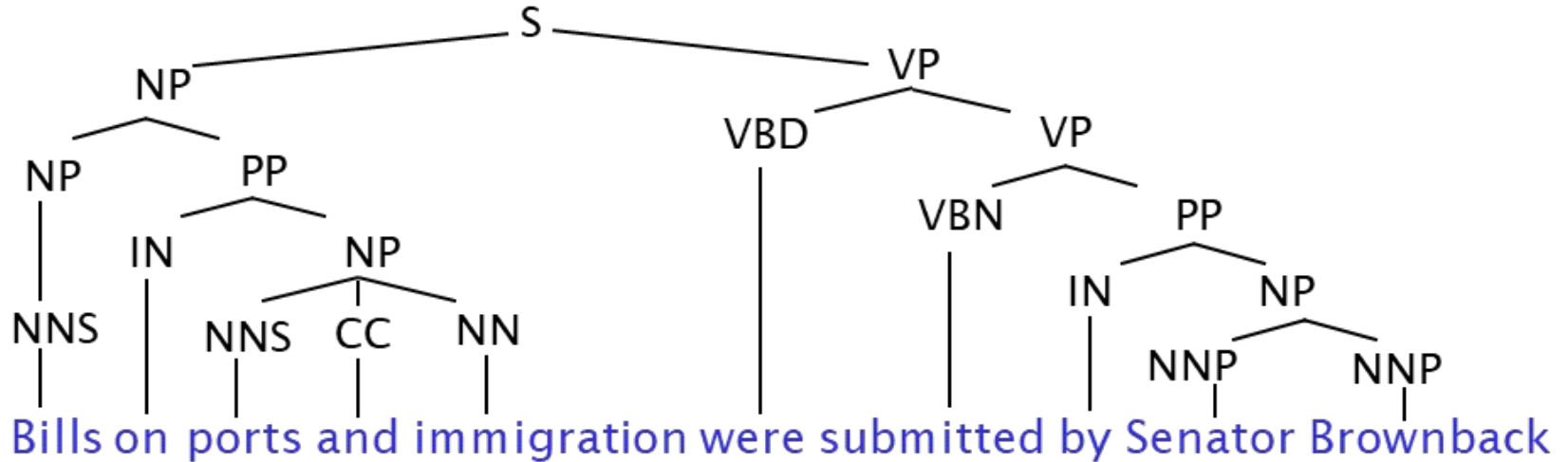
Dependency Parsing

- **Word to word links** instead of constituency
- The original notions of **Subject, Object and sub-categorization** came out of **Dependency theory**.
- Dependency parsing is **quite popular as a computational model** since relationships between words are quite useful
- **Example:** **multi-document summarization**
- Need to **identify sentences from different documents** that each say roughly the same thing
- Phrase structure trees of **paraphrasing sentences which differ in word order can be significantly different**, **but** **dependency representations will be very similar**.

Dependency Parsing: Example

Parse tree:

Nesting of multi-word constituents



Lab Materials : n-Gram Model

- An **n-gram** is a subsequence of n items from a given sequence.
 - Unigram: n -gram of size 1
 - Bigram: n -gram of size 2
 - Trigram: n -gram of size 3

Item:

- Phonemes
- Syllables
- Letters
- Words or Anything else depending on the application.

Lab Materials : n-Gram Model - Example

- the dog smelled like a skunk
- **Bigram:**

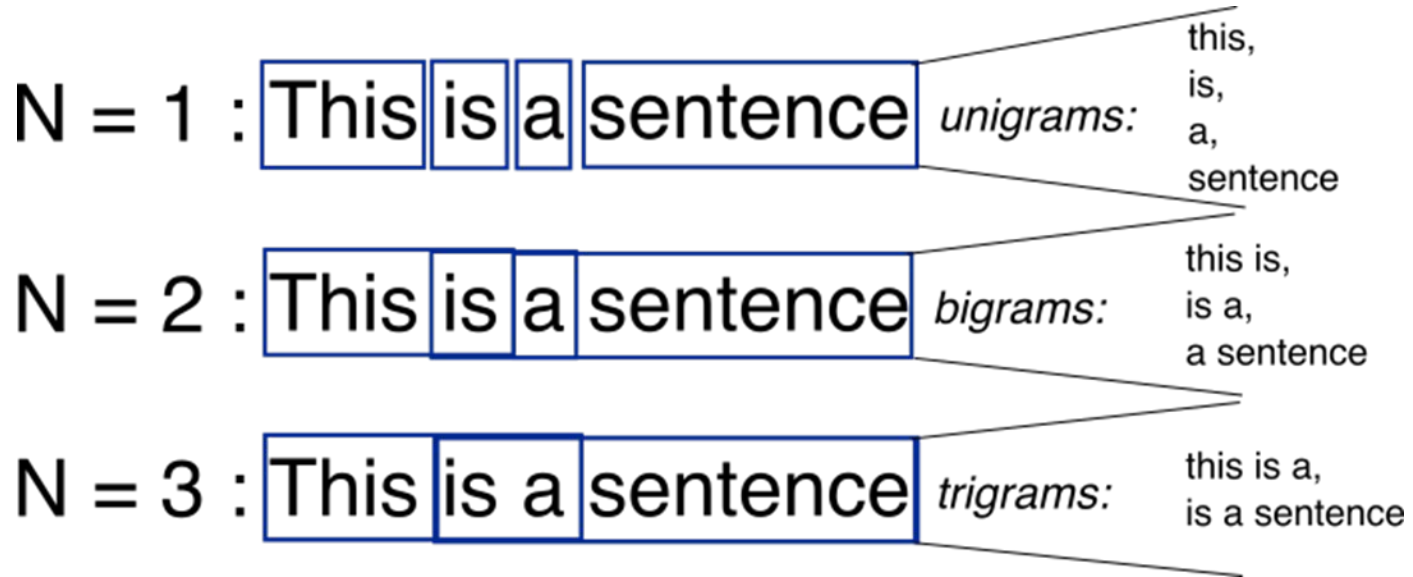
"# the", "the dog", "dog smelled", " smelled like", "like a", "a skunk", "skunk#"

- **Trigram:**

"# the dog", "the dog smelled", "dog smelled like", "smelled like a", "like a skunk" and "a skunk #"

Bag-of-Words with N-grams

- **N-grams**: a contiguous sequence of n tokens from a given piece of text



N-gram Models

- **Unigram model:** $P(w_1)P(w_2)P(w_3)\dots P(w_n)$
- **Bigram model:** $P(w_1)P(w_2 | w_1)P(w_3 | w_2)\dots P(w_n | w_{(n-1)})$
- **Trigram model:**
- $P(w_1)P(w_2 | w_1)P(w_3 | w_2, w_1)\dots P(w_n | w_{(n-1)} w_{(n-2)})$
- **N-gram model:**
- $P(w_1)P(w_2 | w_1)\dots P(w_n | w_{(n-1)} w_{(n-2)}\dots w_{(n-N)})$

N-gram Models : Example

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s>I am nere </s>

<s>who am I </s>

<s>I would like to know </s>

Estimating bigrams

$$P(I | <s>) = 2/3$$

$$P(</s> | here) = 1$$

$$P(would | I) = 1/3$$

$$P(here | am) = 1/2$$

$$P(know | like) = 0$$

N-gram Models : Example

- Estimate the likelihood of the sentence : **I want to eat Chinese food.**
- $P(\text{I want to eat Chinese food}) = P(\text{I} \mid \text{<start>}) P(\text{want} \mid \text{I}) P(\text{to} \mid \text{want})$
 $P(\text{eat} \mid \text{to}) P(\text{Chinese} \mid \text{eat}) P(\text{food} \mid \text{Chinese}) P(\text{<end>} \mid \text{food})$
- What do we need to calculate these likelihoods?
- **Bigram probabilities for each word pair sequence in the sentence**
- Calculated from a large corpus

Early Bigram Probabilities from BERP

Eat on	.16	Eat Thai	.03
Eat some	.06	Eat breakfast	.03
Eat lunch	.06	Eat in	.02
Eat dinner	.05	Eat Chinese	.02
Eat at	.04	Eat Mexican	.02
Eat a	.04	Eat tomorrow	.01
Eat Indian	.04	Eat dessert	.007
Eat today	.03	Eat British	.001

Early Bigram Probabilities from BERP

<start> I	.25	Want some	.04
<start> I'd	.06	Want Thai	.01
<start> Tell	.04	To eat	.26
<start> I'm	.02	To have	.14
I want	.32	To spend	.09
I would	.29	To be	.02
I don't	.08	British food	.60
I have	.04	British restaurant	.15
Want to	.65	British cuisine	.01
Want a	.05	British lunch	.01

Early Bigram Probabilities from BERP

- $P(\text{I want to eat British food}) = P(\text{I}|\text{<start>}) P(\text{want}|\text{I}) P(\text{to}|\text{want}) P(\text{eat}|\text{to}) P(\text{British}|\text{eat}) P(\text{food}|\text{British}) = .25*.32*.65*.26*.001*.60 = .000080$
- Suppose $P(\text{<end>}|\text{food}) = .2$?
- Calculate : I want to eat Chinese food & I Don't eat in British Restaurant.
- Probabilities roughly capture ``syntactic'' facts and ``world knowledge''
eat is often followed by an NP
- British food is not too popular
- **N-gram models can be trained by counting and normalization**